HW4 Approach Component

Goal

Build the **minimum viable submission** that satisfies the grader:

- one importable Python module
- exact required function names
- correct bitmap schema
- working WAH compression for 8, 16, 32, 64
- required output naming
- README + comparison counts

Ignore BBC. Ignore overengineering. Ignore elegance beyond "clean enough to defend."

Core burn-into-head facts

Bitmap facts

- There are **16 total bitmap columns**:
  - 4 Animal
  - 10 Age bins
  - 2 Adopted
- Each tuple contributes:
  - **1 one** in Animal group

    - **1 one** in Age group
    - **1 one** in Adopted group
- Everything else for that tuple is 0
- Column order is fixed:
    - cat, dog, turtle, bird
    - 1–10, 11–20, ..., 91–100
    - True, False

WAH facts

- Compress **each bitmap column independently**
- payload_size = word_size - 1
- A chunk is:
    - **literal** if mixed
    - **fill/run** if all 0s or all 1s
- Even a **single all-0 or all-1 chunk** should be encoded as a run
- Final short chunk:
    - pad on the **right with 0s**
    - encode as literal if needed
- Output is written as **characters**, not true packed machine bits

Assignment facts

- Required functions:
    - create_index(input_file, output_path, sorted)
    - compress_index(bitmap_index, output_path, compression_method, word_size)
- sorted mainly controls filename flag
- You do **not** need BBC
- The grader will likely **import your file and call the functions directly**

Minimum viable philosophy

This project is not about making a perfect compression library.

This project is about:

1. matching the spec
2. producing correct files
3. being able to explain your structure in simple terms

Your standard should be:

"It is plain, correct enough, and traceable to the handout/slides."

Project phases

Phase 0 — Setup and lock the shape

Objective

Create the exact file and function skeleton so you stop spiraling.

Deliverable

One Python file, likely hw4.py, containing:

- constants for domains
- required function stubs
- helper stubs

Bare minimum structure

```
ANIMALS = ['cat', 'dog', 'turtle', 'bird']
AGE_BINS = [(1,10), (11,20), ..., (91,100)]

def create_index(input_file, output_path, sorted):
    pass

def compress_index(bitmap_index, output_path, compression_method,
word_size):
    pass
```

Done when

- file imports without crashing
- function names match spec exactly

Spiral blocker

Do **not** think about WAH yet.

Phase 1 — Input parsing

Objective

Read the raw data file and turn each line into usable Python values.

What to implement

- parse_row(line)
- parse_bool(value)

Input assumption

Each row has:

- Animal
- Age
- Adopted

Decisions to lock

- delimiter
- trimming whitespace
- True/False parsing
- lowercase animal normalization

Done when

Given one row, you can get:

('cat', 27, True)

Burn-into-head facts

- Don’t guess beyond the file format
- Parsing first reduces later confusion
- If parsing is shaky, everything downstream lies

Phase 2 — Domain mapping

Objective

Map each tuple into the correct bitmap positions.

What to implement

- animal_index(animal)
- age_bin_index(age)
- adopted_index(adopted)

Expected mapping

Animal

- cat -> 0
- dog -> 1
- turtle -> 2
- bird -> 3

Age bins

- 1–10 -> 0
- 11–20 -> 1
- …
- 91–100 -> 9

Adopted

- True -> 0
- False -> 1

Done when

For any parsed tuple, you can say:

- which Animal bitmap gets the 1
- which Age bitmap gets the 1
- which Adopted bitmap gets the 1

Spiral blocker

This phase is just **mapping**, not file writing.

Phase 3 — Build raw bitmap index

Objective

Construct the 16 bitmap columns correctly.

Recommended representation

Use a list of 16 lists of characters:

bitmaps = [[], [], ..., []]

Reason:

- easy to append '0'
- easy to flip one final position to '1'
- easy to join into strings later

What to implement

- initialize_bitmaps()
- append_zeroes_to_all(bitmaps)
- set_tuple_bits(bitmaps, animal, age, adopted)
- flatten_bitmaps(bitmaps) if you use grouped storage

Algorithm

For each tuple:

1. append '0' to all 16 columns
2. find 3 target columns
3. change the newest bit in those 3 columns to '1'

Why this works

It directly follows:

- one 1 per attribute group
- zeroes elsewhere

Done when

For n tuples:

- each bitmap has length n
- each row contributes exactly 3 ones total across all 16 columns

Validation test

For 1 tuple:

- total ones across all bitmaps = 3

For 2 tuples:

- total ones across all bitmaps = 6

Spiral blocker

Do **not** compress yet. Just print the 16 raw bitmaps and verify shape.

Phase 4 — Write bitmap index file

Objective

Save the raw bitmap index in the expected output form.

What to implement

- build_index_filename(input_file, sorted)
- write_bitmap_file(bitmaps, output_file)

Practical approach

Write one bitmap column per line as a string of 0 and 1.

That keeps things simple and inspectable.

Filename goal

Use the current input filename and append flags only as required.

Done when

Running create_index(...) produces:

- a real file
- correct filename pattern
- 16 lines / 16 bitmap strings

Validation test

Open the output file and verify:

- 16 bitmap entries exist
- each has length equal to number of tuples

Spiral blocker

Don’t optimize the file format. Clarity beats cleverness.

Phase 5 — Lock create_index

Objective

Finish one entire required function before touching compression.

create_index should now do

1. read file
2. parse rows
3. build bitmaps
4. write bitmap file
5. return useful path or finish cleanly

Done when

You can call:

create_index("inputfile", ".", False)

and it generates a bitmap index file correctly.

Minimum confidence target

You should be able to explain:

"I turned rows into 16 fixed bitmap columns using one-hot positions per attribute."

If yes, move on.

Phase 6 — Read bitmap index back in

Objective

Prepare compression input cleanly.

What to implement

- read_bitmap_file(bitmap_index)

Goal

Return the 16 bitmap strings from the saved file.

Done when

You can read the file and get:

['001010...', '110000...', ...]

Spiral blocker

No compression logic yet. Just loading.

Phase 7 — WAH chunking

Objective

Convert one bitmap string into payload chunks.

What to implement

- chunk_bitmap(bitmap, payload_size)
- is_all_zero(chunk)
- is_all_one(chunk)

Rule

payload_size = word_size - 1

Examples:

- word size 8 → payload 7
- word size 16 → payload 15
- word size 32 → payload 31
- word size 64 → payload 63

Final short chunk rule

If the last chunk is too short:

- pad on the **right with 0s**

Done when

Given a bitmap string and word size, you can print the exact payload chunks.

Validation test

If bitmap length is not divisible by payload size:

- last chunk still exists
- last chunk has exact payload length after padding

Burn-into-head facts

- chunking is the bridge between bitmap and WAH
- if chunking is wrong, every later output is wrong

Phase 8 — WAH encoding for one bitmap

Objective

Encode one bitmap column correctly before generalizing to all 16.

What to implement

- encode_literal(chunk)
- encode_fill(fill_bit, run_count, word_size)
- compress_bitmap_wah(bitmap, word_size)

Minimum mental model

For each payload chunk:

- all 0 → zero-fill run
- all 1 → one-fill run
- mixed → literal

Simple implementation plan

Walk left to right through chunks:

1. inspect current chunk
2. if all-0, count consecutive all-0 chunks
3. if all-1, count consecutive all-1 chunks
4. otherwise emit one literal

Track counts

Inside compress_bitmap_wah:

- fill_words += 1 whenever you emit a fill word
- literal_words += 1 whenever you emit a literal

Done when

One bitmap can be compressed and returns:

- encoded words
- fill count
- literal count

Validation test

Use tiny hand-made examples:

- all zero bitmap
- all one bitmap
- mixed bitmap
- bitmap with leftover short tail

Spiral blocker

Get **8-bit** working first. Then generalize.

Phase 9 — Generalize WAH to all required word sizes

Objective

Make the same logic work for 8, 16, 32, 64.

What changes

Only:

- payload_size
- run-count field width inside your fill encoding

What should not change

- overall algorithm
- chunk classification
- literal vs fill decision

Done when

You can run:

```
compress_bitmap_wah(bitmap, 8)
compress_bitmap_wah(bitmap, 16)
compress_bitmap_wah(bitmap, 32)
compress_bitmap_wah(bitmap, 64)
```

Burn-into-head facts

- same logic, different sizes
- don’t fork four separate implementations

Phase 10 — Compress full bitmap index

Objective

Finish compress_index(...).

What to implement

- build_compressed_filename(...)
- write_compressed_file(...)
- full loop over all 16 bitmap columns

compress_index

should now do

1. read bitmap file
2. verify compression method is "WAH"
3. compress each bitmap independently
4. accumulate total fill/literal counts
5. write compressed output file
6. return stats if useful

Done when

Calling:

compress_index(bitmap_index_file, ".", "WAH", 32)

produces:

- compressed output file
- correct filename
- fill/literal counts

Spiral blocker

Ignore BBC completely.

Phase 11 — Generate required outputs

Objective

Produce all artifacts needed for the writeup.

Minimum output set

You need enough to compare:

- unsorted bitmap index
- sorted-flag bitmap index
- WAH compressed outputs for both
- for word sizes 8, 16, 32, 64

Practical note

If true sorting is not required, then the sorted flag may only affect filename. Follow the assignment wording exactly.

What to collect

For each produced file:

- file size
- fill word count
- literal word count

Done when

You have a table of numbers ready for README/writeup.

Phase 12 — README and writeup

Objective

Finish the final 30% without overthinking.

README should include

- what files are included
- what each file does
- how to run create_index
- how to run compress_index
- what output files get produced

Writeup should answer

1. Why do sizes differ?
2. Did sorting help?
3. By how much?
4. How does word size affect compression?
5. How many fill words and literal words were produced?

Safe explanation patterns

Why sizes differ

- bitmap columns differ in how many long repeated runs they contain

Why sorting helps

- sorting tends to cluster similar values, which increases run lengths

Why word size matters

- larger payloads change how often chunks become fills versus literals
- different word sizes change the balance between overhead and run grouping

Done when

Your writeup uses your actual collected numbers.

Suggested helper list

Core helpers

- parse_row(line)
- parse_bool(value)
- animal_index(animal)
- age_bin_index(age)
- adopted_index(adopted)

Bitmap helpers

- initialize_bitmaps()
- append_zeroes_to_all(bitmaps)
- set_tuple_bits(bitmaps, animal, age, adopted)
- write_bitmap_file(bitmaps, output_file)
- read_bitmap_file(path)

Naming helpers

- build_index_filename(input_file, sorted)
- build_compressed_filename(bitmap_index, compression_method, word_size)

WAH helpers

- chunk_bitmap(bitmap, payload_size)

- is_all_zero(chunk)
- is_all_one(chunk)
- encode_literal(chunk)
- encode_fill(fill_bit, run_count, word_size)
- compress_bitmap_wah(bitmap, word_size)
- write_compressed_file(encoded_words, output_file)

Development order

Use this exact order and do not skip ahead:

1. file skeleton
2. parsing
3. domain mapping
4. raw bitmap build
5. bitmap file writing
6. finish create_index
7. bitmap file reading
8. chunking
9. one-bitmap WAH with 8-bit
10. generalize to 16/32/64
11. finish compress_index
12. collect stats
13. README/writeup
14. final packaging

Time-boxed 1–2 day plan

Day 1

Block 1 — 45 min

Set up file skeleton, constants, parsing helpers

Block 2 — 60–90 min

Implement domain mapping + bitmap construction

Block 3 — 45 min

Write bitmap file and finish create_index

Block 4 — 60–90 min

Implement chunking + test by printing chunks

Block 5 — 60–90 min

Implement WAH for one bitmap, 8-bit only

Day 2

Block 6 — 45–60 min

Generalize WAH to 16/32/64

Block 7 — 45–60 min

Finish compress_index

Block 8 — 45 min

Generate outputs + collect file sizes and counts

Block 9 — 45–60 min

README + writeup + zip packaging

Anti-spiral rules

Rule 1

Do not touch BBC.

Rule 2

Do not build classes unless forced.

Rule 3

Do not pack true binary bytes unless the handout absolutely forces it.

Rule 4

Do not optimize before create_index and compress_index both run end-to-end.

Rule 5

Whenever stuck, return to these 3 questions:

- What is one input row?
- Which 3 bitmap positions get 1?
- Is this payload chunk literal or fill?

Quick self-check checkpoints

Checkpoint A — after bitmap builder

Can you answer:

- how many columns exist?
- why exactly 3 ones per tuple?
- which age bin does 27 belong to?

If no, stop and fix.

Checkpoint B — after chunking

Can you answer:

- for word size 8, why is payload size 7?

- what happens to leftover 3 bits?

If no, stop and fix.

Checkpoint C — after one-bitmap WAH

Can you answer:

- when do I emit a literal?
- when do I emit a fill?
- why does a single all-0 chunk still become a run?

If no, stop and fix.

Checkpoint D — before submission

Can you:

- import the file?
- call both required functions?
- produce files with correct names?
- report file size + fill/literal counts?

If yes, you are basically done.

Definition of done

You are done when:

1. hw4.py imports without errors
2. it exposes:
    a. create_index(input_file, output_path, sorted)
    b. compress_index(bitmap_index, output_path, compression_method, word_size)
3. bitmap output has 16 bitmap columns
4. WAH works for 8, 16, 32, 64
5. output filenames follow the required pattern
6. you can report:
    a. compressed size
    b. fill words
    c. literal words
7. README explains how to run it
8. writeup answers the comparison questions

Final mindset

This assignment is not:

- “master bitmap compression theory”

It is:

- “implement the spec plainly, verify the pipeline, report the results”

When you feel yourself spiraling, burn this into head:

- 16 columns
- 3 ones per tuple
- payload is word_size - 1
- mixed = literal
- all-0/all-1 = fill
- leftover tail pads right with 0
- make the files
- count the words
- move on

If you want, next I'll turn this into a **copy-paste README approach section** in the same tone, ready to drop into your submission.